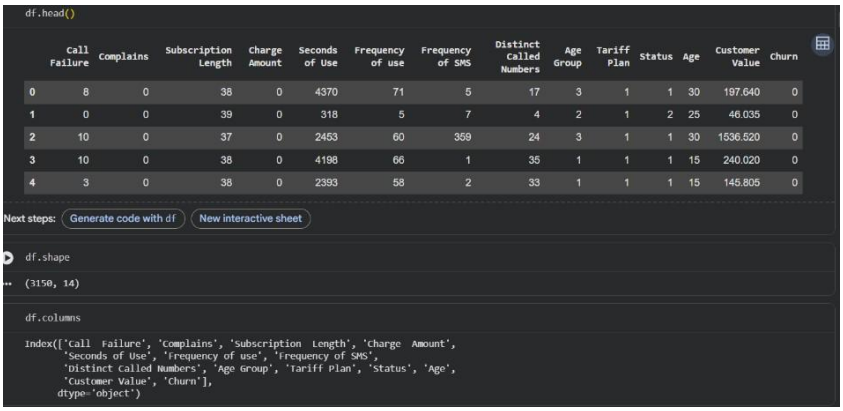
Customer Churn Prediction Using Machine Learning

Name: Karunanithy Danchika

This project focuses on predicting customer churn using two machine learning models: XGBoost and Multilayer Perceptron (MLP). The main objective is to develop and compare the performance of both models for customer churn classification. The models are evaluated using accuracy, precision, recall, and F1-score.

Dataset and Preprocessing

The customer churn dataset was loaded into a Pandas DataFrame. Initial data exploration was performed using the dataset shape, column names, data types, descriptive statistics, missing-value checks, duplicate checks, and churn class distribution. The target variable was identified as Churn, while the remaining columns were used as input features.



```
df.head()
```

	Call Failure	Complains	Subscription Length	Charge Amount	Seconds of Use	Frequency of use	Frequency of SMS	Distinct Called Numbers	Age Group	Tariff Plan	Status	Age	Customer Value	Churn
0	8	0	38	0	4370	71	5	17	3	1	1	30	197.640	0
1	0	0	39	0	318	5	7	4	2	1	2	25	46.035	0
2	10	0	37	0	2453	60	359	24	3	1	1	30	1536.520	0
3	10	0	38	0	4198	66	1	35	1	1	1	15	240.020	0
4	3	0	38	0	2393	58	2	33	1	1	1	15	145.805	0

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(3150, 14)
```

```
df.columns
```

```
Index(['Call  Failure', 'Complains', 'Subscription  Length', 'Charge  Amount',
       'Seconds of Use', 'Frequency of use', 'Frequency of SMS',
       'Distinct Called Numbers', 'Age Group', 'Tariff Plan', 'Status', 'Age',
       'Customer Value', 'Churn'],
      dtype='object')
```



```
Dataset Shape: (3150, 14)

Column Names:
['Call  Failure', 'Complains', 'Subscription  Length', 'Charge  Amount', 'Seconds of Use', 'Frequency of use', 'Frequency of SMS', 'Distinct Ca

Missing Values:
Call  Failure             0
Complains                 0
Subscription  Length      0
Charge  Amount            0
Seconds of Use            0
Frequency of use          0
Frequency of SMS          0
Distinct Called Numbers   0
Age Group                 0
Tariff Plan               0
Status                    0
Age                       0
Customer Value            0
Churn                     0
dtype: int64

Duplicate Rows:
300

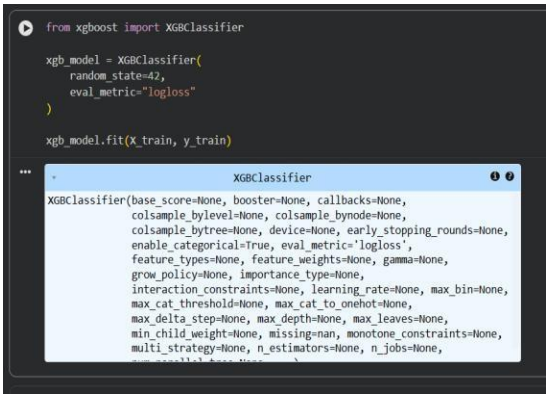
Churn Distribution:
Churn
0    2655
1     495
Name: count, dtype: int64
```

Model Development

Two machine learning models, XGBoost and Multilayer Perceptron (MLP), were developed for customer churn prediction. Hyperparameter optimization was performed for both models to improve their classification performance.

XGBoost Model

XGBoost was used as a tree-based classification model for predicting customer churn. The model was optimized using RandomizedSearchCV. The hyperparameter search included the number of estimators, maximum tree depth, learning rate, subsampling, and column sampling. Five-fold cross-validation was used and F1-score was selected as the optimization metric.

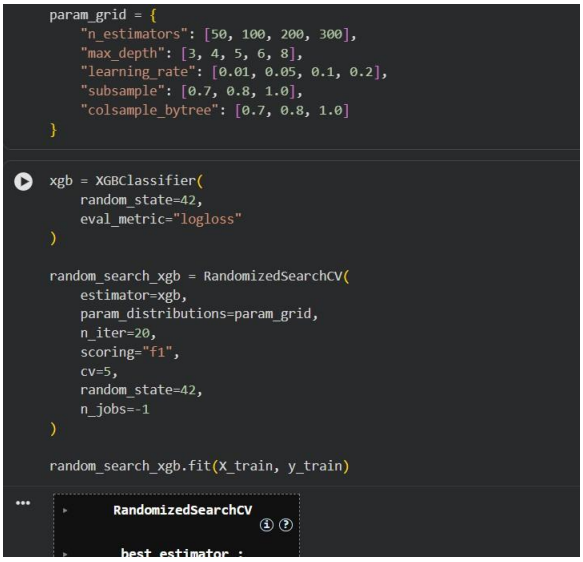


```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    random_state=42,
    eval_metric="logloss"
)

xgb_model.fit(X_train, y_train)
```

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=True, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
```



```
param_grid = {
    "n_estimators": [50, 100, 200, 300],
    "max_depth": [3, 4, 5, 6, 8],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "subsample": [0.7, 0.8, 1.0],
    "colsample_bytree": [0.7, 0.8, 1.0]
}
```

```
xgb = XGBClassifier(
    random_state=42,
    eval_metric="logloss"
)

random_search_xgb = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_grid,
    n_iter=20,
    scoring="f1",
    cv=5,
    random_state=42,
    n_jobs=-1
)

random_search_xgb.fit(X_train, y_train)
```

```
RandomizedSearchCV
best_estimator_:
```

Multilayer Perceptron (MLP)

A Multilayer Perceptron neural network was developed using TensorFlow/Keras. The network contains dense layers with ReLU activation, dropout layers for regularization, and a sigmoid output layer for binary classification. The MLP was trained using the Adam optimizer and binary cross-entropy loss.

MLP Hyperparameter Optimization

Hyperparameter optimization was performed by testing different combinations of neurons, dropout rate, learning rate, batch size, and number of epochs. Ten randomly selected combinations were evaluated, and the configuration with the highest validation F1-score was selected as the best MLP configuration.

Performance Evaluation

The performance of the XGBoost and MLP models was evaluated using four classification metrics: accuracy, precision, recall, and F1-score. These metrics were calculated on the test dataset to compare the predictive performance of the two models.

Performance Metrics Table

========== FINAL MODEL COMPARISON ==========

Metric	XGBoost	MLP
Accuracy	0.971429	0.949206
Precision	0.893204	0.924051
Recall	0.929293	0.737374
F1-score	0.910891	0.820225

Classification Report screenshot



```
from sklearn.metrics import classification_report

print("XGBoost Classification Report")

print(
    classification_report(
        y_test,
        y_pred_xgb_best
    )
)
```

```
XGBoost Classification Report
              precision    recall  f1-score   support

           0       0.98      0.97      0.98       531
           1       0.86      0.88      0.87        99

    accuracy                           0.96       630
   macro avg       0.92      0.93      0.92       630
weighted avg       0.96      0.96      0.96       630
```

```
from sklearn.metrics import classification_report

print("MLP Classification Report")

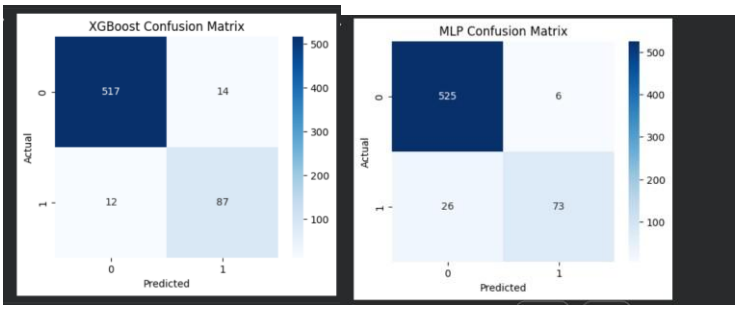
print(
    classification_report(
        y_test,
        y_pred_mlp,
        zero_division=0
    )
)
```

```
MLP Classification Report
              precision    recall  f1-score   support

           0       0.95      0.99      0.97       531
           1       0.92      0.74      0.82        99

    accuracy                           0.95       630
   macro avg       0.94      0.86      0.90       630
weighted avg       0.95      0.95      0.95       630
```

Confusion Matrices



Accuracy represents the overall proportion of correctly classified customers. Precision indicates how accurately the models identify customers predicted as churners. Recall measures the ability to identify actual churn cases, while F1-score provides a balance between precision and recall.

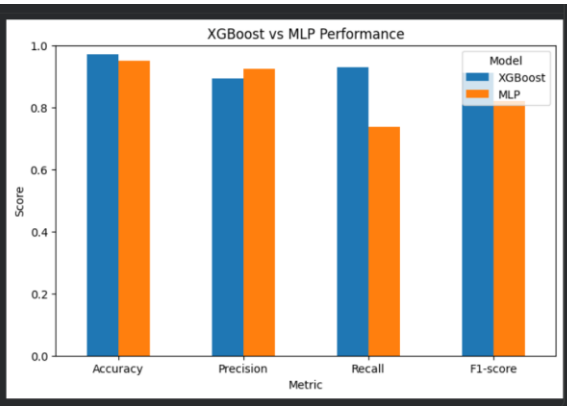
Model Comparison & Conclusion

```
comparison_plot = comparison.set_index("Metric")

comparison_plot.plot(
    kind="bar",
    figsize=(8,5)
)

plt.title("XGBoost vs MLP Performance")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(title="Model")

plt.show()
```



Discussion

The results show that both models were able to classify customer churn using the selected features. The comparison of accuracy, precision, recall, and F1-score provides a broader understanding of model performance. F1-score is particularly useful for comparing the models because it considers both precision and recall. The model with the higher F1-score can therefore be considered the better-performing model for this classification task.

Conclusion

In conclusion, XGBoost and MLP were successfully implemented for customer churn prediction. Hyperparameter optimization was applied to improve the models, and their performance was evaluated using accuracy, precision, recall, and F1-score. Based on the final evaluation results, the model with the stronger F1-score demonstrated better overall performance for the customer churn prediction task.